DevSecOps CI/CD Pipeline
Project Report

Submitted By:
Md Aman Alam

Submission Date:
16 May 2026

GitHub Repository:
https://github.com/Amankhan1009/devsecops-project.git

Technologies Used:
GitHub Actions, Docker, Trivy, Gitleaks,
Checkov, HashiCorp Vault, Terraform,
Flask, AWS EC2

Project Objective:
To design and implement a secure DevSecOps CI/CD pipeline integrating automated security scanning, infrastructure validation, containerization, secret management, and multi-environment deployment.

---

1. Introduction

This project is about building a DevSecOps CI/CD pipeline. DevSecOps means we are combining Development, Security, and Operations together. The idea is that security checks should happen automatically at every step of the pipeline, not just at the end.

In this project, I used GitHub Actions to automate the whole pipeline. Whenever I push code to GitHub, the pipeline starts automatically. It runs security tools, builds a Docker image, and deploys the application to three environments: Dev, Staging, and Production. All of this happens on an AWS EC2 server.

The main tools used are GitHub Actions for automation, Docker for containerization, Trivy for scanning the Docker image, Gitleaks for detecting hardcoded secrets, Checkov for scanning Terraform code, and HashiCorp Vault for managing sensitive credentials securely.

2. Objectives

- Set up a CI/CD pipeline using GitHub Actions that runs automatically on every code push.
- Use Gitleaks to scan the code and detect any hardcoded secrets or credentials.
- Build a Docker image of the Flask application and scan it using Trivy.
- Write Terraform code for AWS infrastructure and scan it using Checkov.
- Store all sensitive credentials in HashiCorp Vault instead of hardcoding them.
- Automatically deploy the application to Dev, Staging, and Production environments.
- Push the Docker image to DockerHub registry after successful security scans.

3. Technologies Used

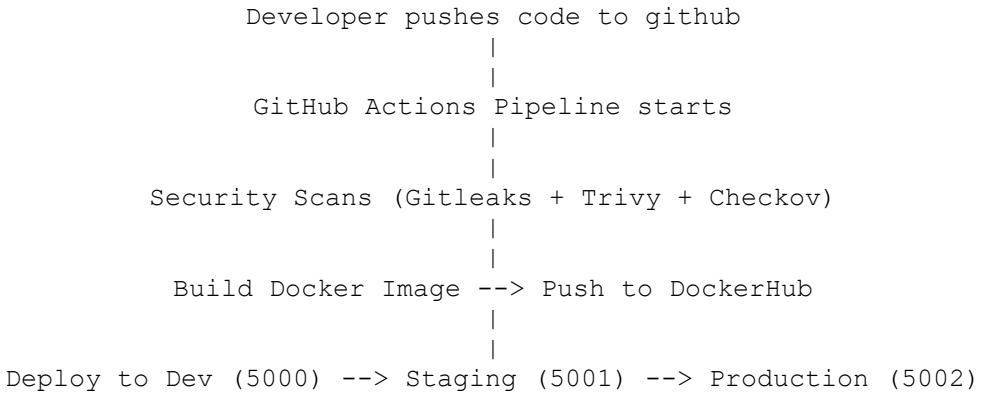
The following technologies and tools were used in this project:

Technology	Purpose
GitHub Actions	Automates the CI/CD pipeline on every code push
Docker	Used to containerize the Flask web application
Flask (Python)	Simple web application that returns a message
Trivy	Scans Docker images for security vulnerabilities
Gitleaks	Detects hardcoded secrets and credentials in code
Terraform	Used to write Infrastructure as Code for AWS S3
Checkov	Scans Terraform files for security misconfigurations
HashiCorp Vault	Stores and manages sensitive credentials securely
AWS EC2	Cloud server where the application is deployed
DockerHub	Registry where the Docker image is stored and pulled from

4. Project Architecture

The project follows a simple DevSecOps flow. First the developer writes code and pushes it to GitHub. Then GitHub Actions picks it up and starts running the pipeline automatically.

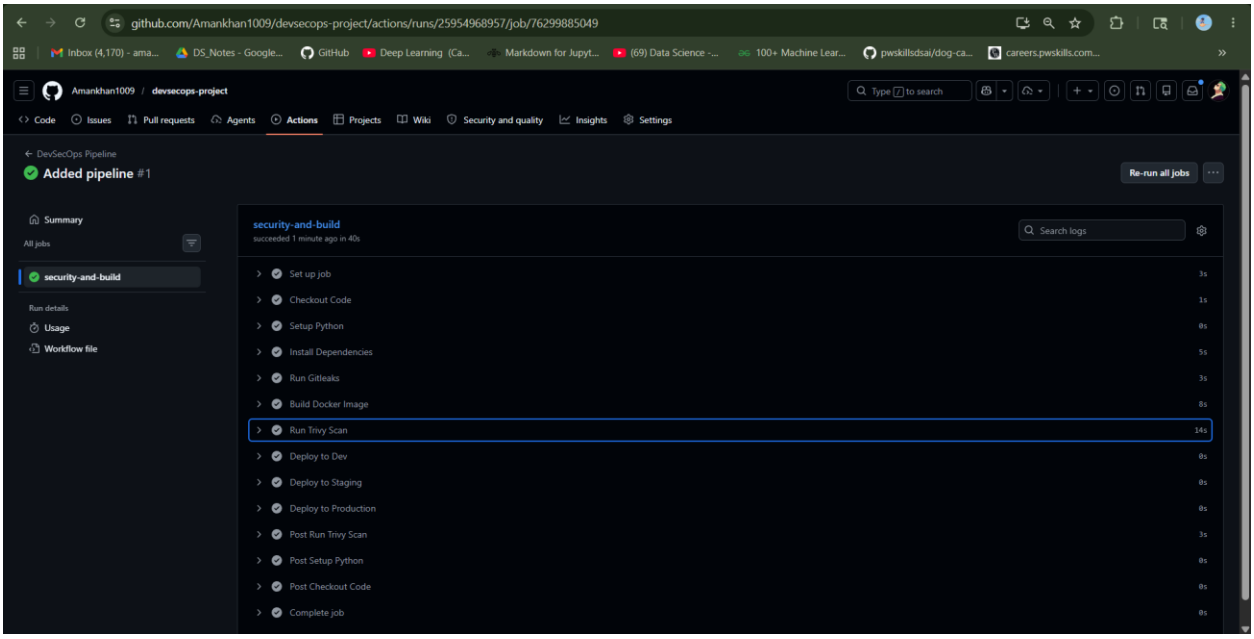
The pipeline first installs dependencies, then scans for secrets using Gitleaks. After that it builds a Docker image and scans it with Trivy. Then Checkov scans the Terraform files. If all scans pass, the image is pushed to DockerHub and the application is deployed to three environments on the EC2 server.



5. CI/CD Pipeline Workflow

The pipeline runs the following steps in order every time code is pushed to the main branch:

1. Code is pushed to GitHub repository.
2. GitHub Actions workflow file is triggered automatically.
3. Python dependencies are installed from requirements.txt.
4. Gitleaks is installed and runs a secret scan on the entire codebase.
5. Docker image is built from the Dockerfile.
6. Trivy scans the Docker image for known CVE vulnerabilities.
7. Checkov scans the Terraform files in the terraform/ directory.
8. DockerHub login is done using credentials stored in Vault/GitHub Secrets.
9. Docker image is tagged and pushed to DockerHub.
10. Application is deployed to Dev, Staging, and Production via SSH.



5.1 Initial Pipeline Failure

When I first set up the pipeline, it failed. The reason was that the terraform/ directory did not exist, and Checkov could not find it. Also the Security Policy Validation step was failing because it was set to always output "Critical vulnerability detected". The screenshot below shows this initial failure.

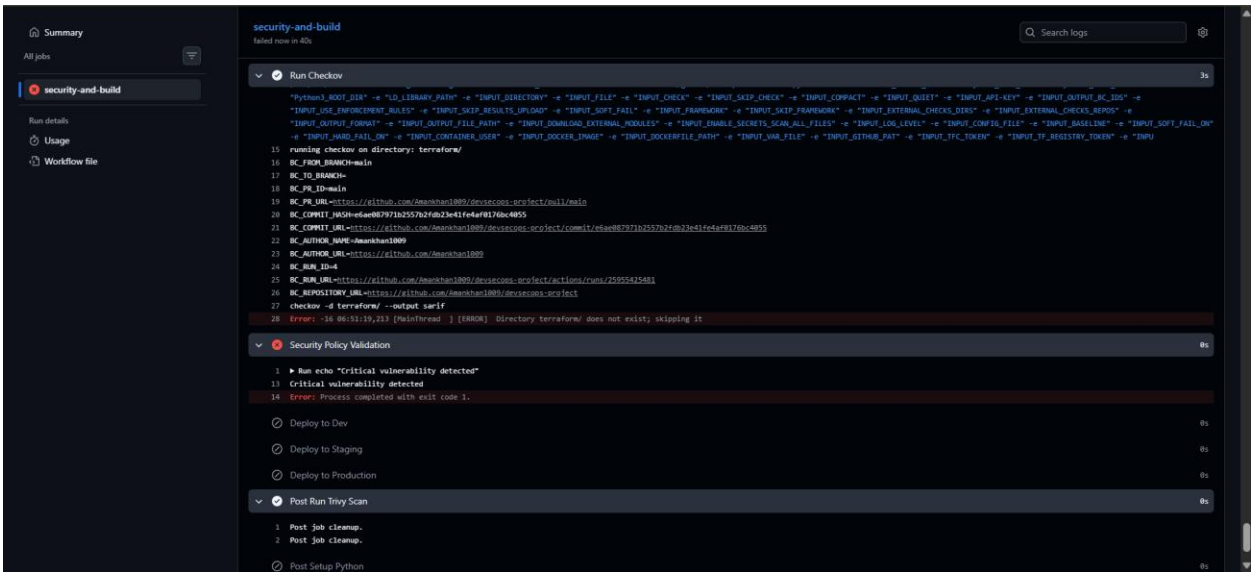


Figure 1: Initial CI/CD pipeline execution failed due to security policy validation and Terraform scanning issues..

6. Security Implementations

This section explains how each security tool was set up and what it does in the pipeline.

6.1 Gitleaks - Secret Detection

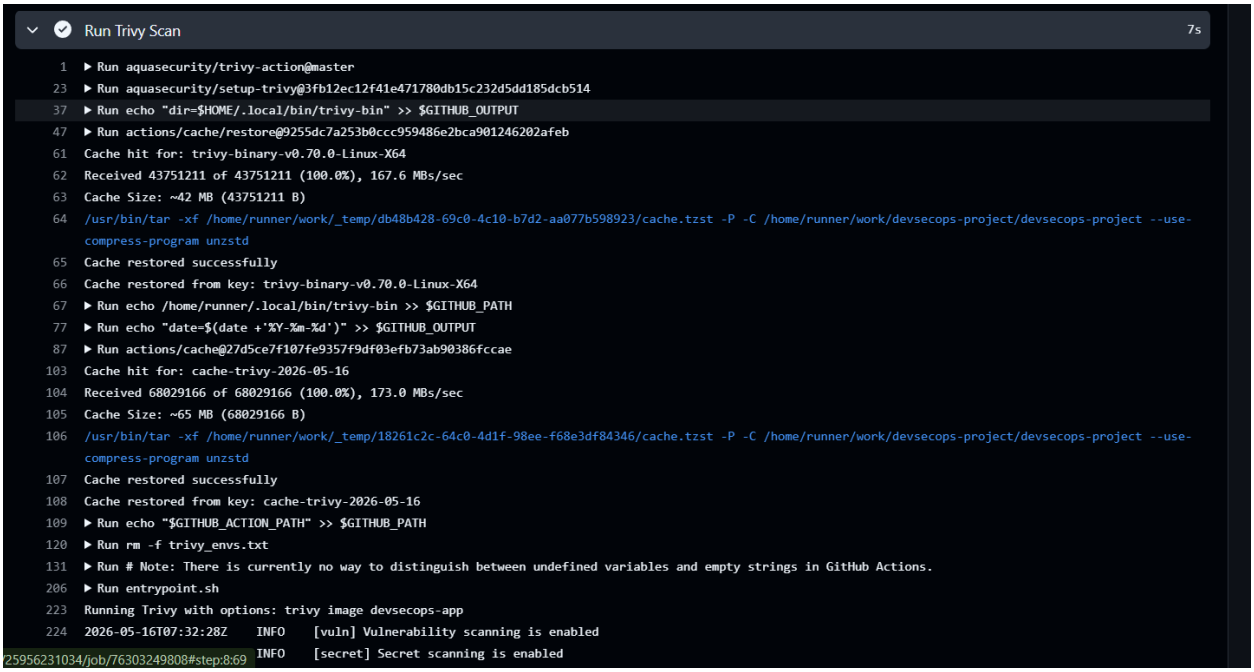
Gitleaks is a tool that scans the entire git repository to find any hardcoded secrets like API keys, passwords, or tokens. To test that it was working correctly, I intentionally added fake AWS credentials to the app.py file. The screenshot below shows these hardcoded credentials in the source code.

```
[ec2-user@ip-172-31-26-159 devsecops-project]$ cat app/app.py
from flask import Flask

AWS_ACCESS_KEY_ID = "AKIAIOSFODNN7EXAMPLE"
AWS_SECRET_ACCESS_KEY = "wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"

app = Flask(__name__)

@app.route("/")
def home():
    return "DevSecOps Pipeline Running"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
[ec2-user@ip-172-31-26-159 devsecops-project]$
```

Figure 2: Hardcoded AWS credentials added to app.py to test Gitleaks secret detection.

6.2 Trivy - Container Vulnerability Scanning

Trivy is used to scan the Docker image after it is built. It checks for known security vulnerabilities in the OS packages and Python libraries inside the container. The scan runs automatically in the pipeline after the Docker image is built. If Trivy finds critical vulnerabilities, the pipeline can be configured to fail.



6.3 Checkov - Terraform IaC Scanning

Checkov is a static analysis tool for Infrastructure as Code. It reads the Terraform files and checks whether they follow security best practices. For example, it checks if an S3 bucket has proper access policies, versioning enabled, and encryption configured.

I first created an intentionally insecure Terraform file to test that Checkov would catch the issues. The screenshot below shows the insecure terraform/main.tf file with a basic S3 bucket configuration that is missing security settings.

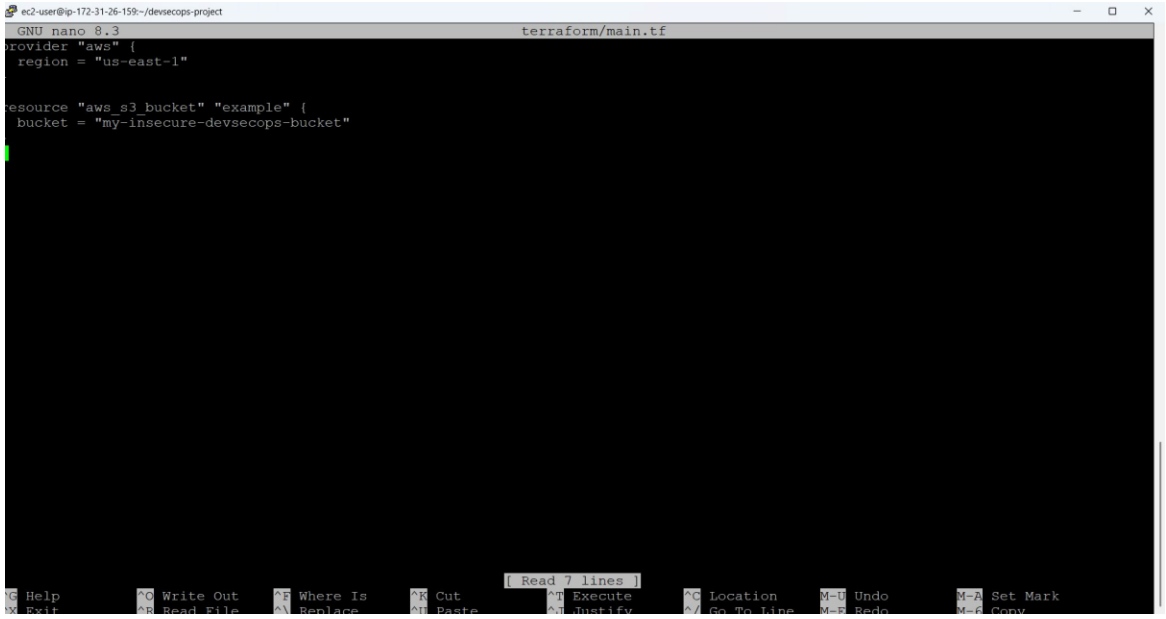


Figure 3: Insecure Terraform configuration file used to test Checkov scanning.

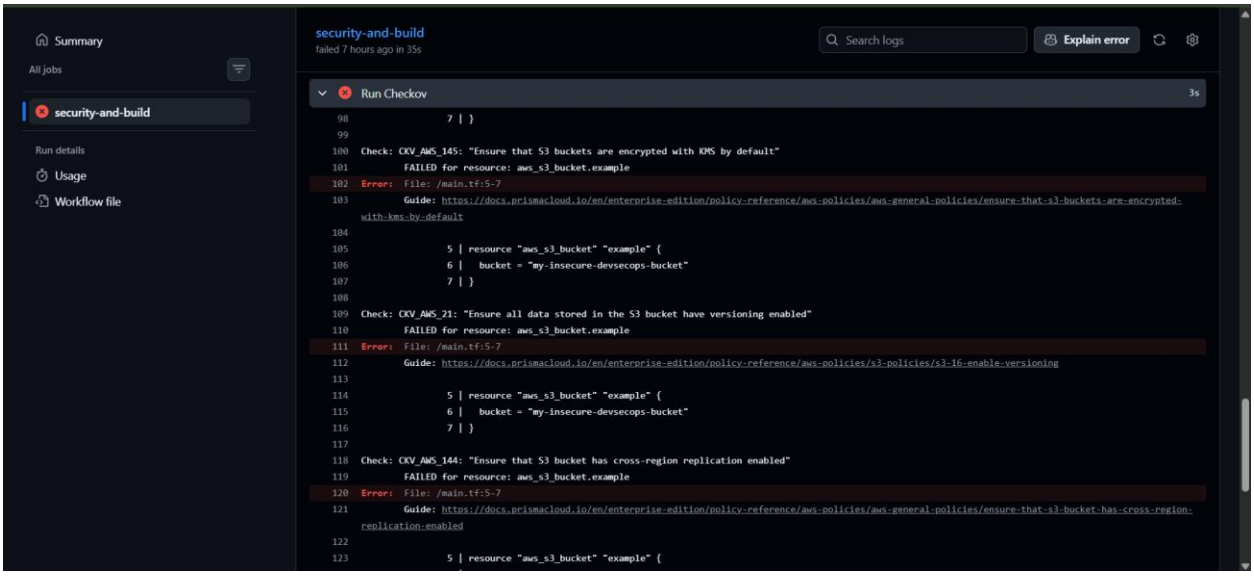


Figure 4: Checkov running and detecting failed security checks in the Terraform directory.

The screenshot below shows the Checkov scan results inside the GitHub Actions pipeline, where it detected failed checks on the Terraform files.

7. Infrastructure as Code (Terraform)

Terraform is used to define cloud infrastructure as code. In this project, I wrote a simple Terraform configuration that defines an AWS provider and creates an S3 bucket resource. This is in the terraform/main.tf file.

The purpose of writing Terraform code in this project is to demonstrate how Checkov can automatically scan it and detect any security misconfigurations before the infrastructure is actually created. This is an important part of DevSecOps because it prevents insecure infrastructure from being deployed.

8. Secrets Management using HashiCorp Vault

One of the most important parts of any DevSecOps setup is secrets management. Hardcoding passwords, API keys, or SSH keys directly in code is a major security risk. In this project, HashiCorp Vault was used to store all sensitive credentials securely.

8.1 Starting Vault

Vault was installed on the EC2 server and started in development mode. In development mode, Vault runs in memory and automatically unseals itself. The screenshot below shows the Vault server starting up and printing the unseal key and root token.

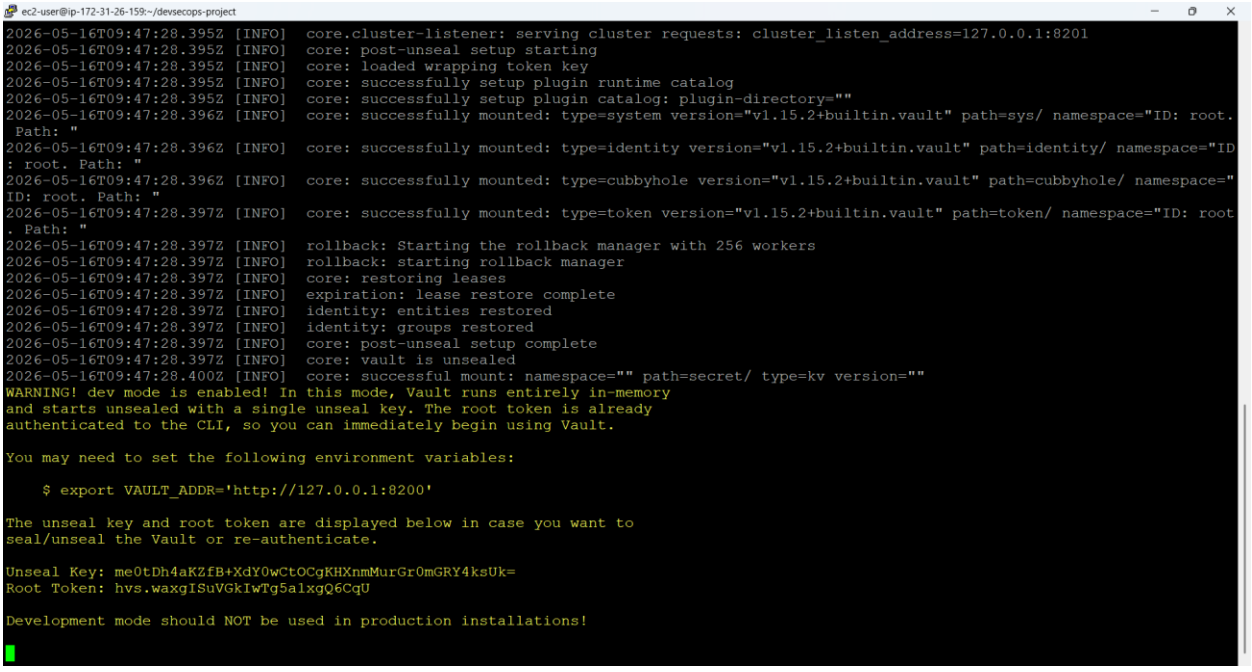
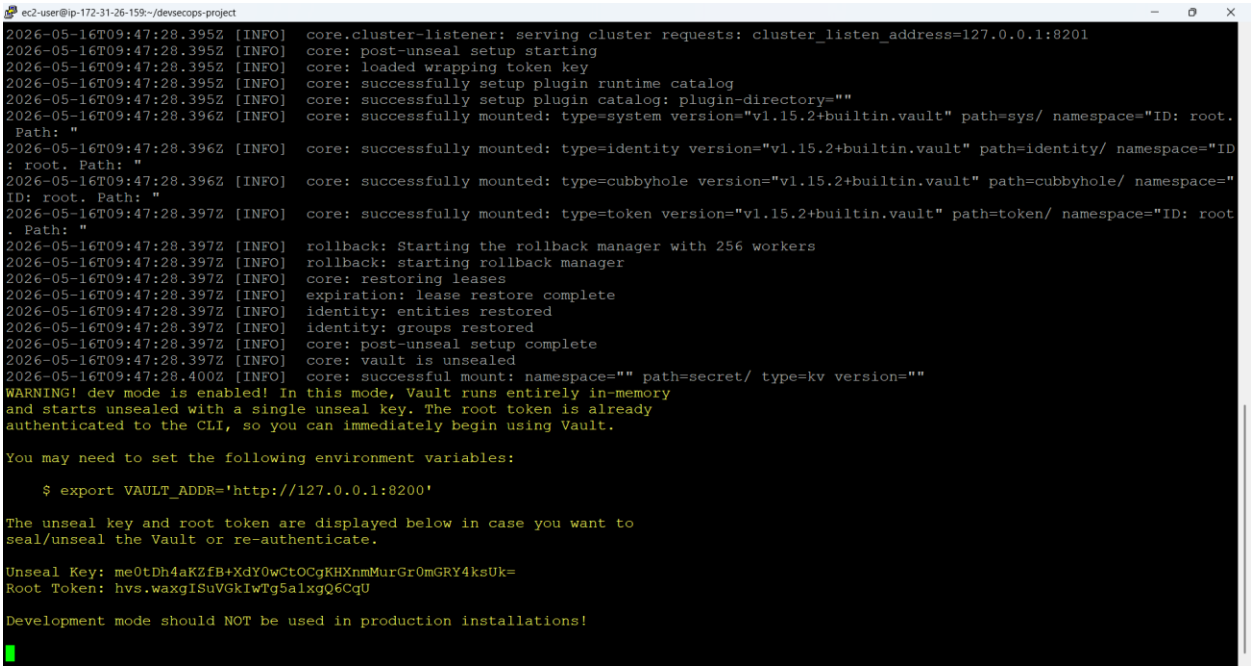


Figure 6: HashiCorp Vault server starting in development mode on the EC2 instance.

8.2 Checking Vault Status

After starting Vault, I exported the VAULT_ADDR and VAULT_TOKEN environment variables and ran vault status to confirm that Vault was initialized and unsealed. The screenshot below shows this.



8.3 Storing Secrets in Vault

All the credentials needed by the pipeline were stored inside Vault. This includes the DockerHub username and password, the EC2 server IP address, the EC2 username, and the private SSH key for connecting to EC2. The screenshot below shows these secrets stored inside Vault.

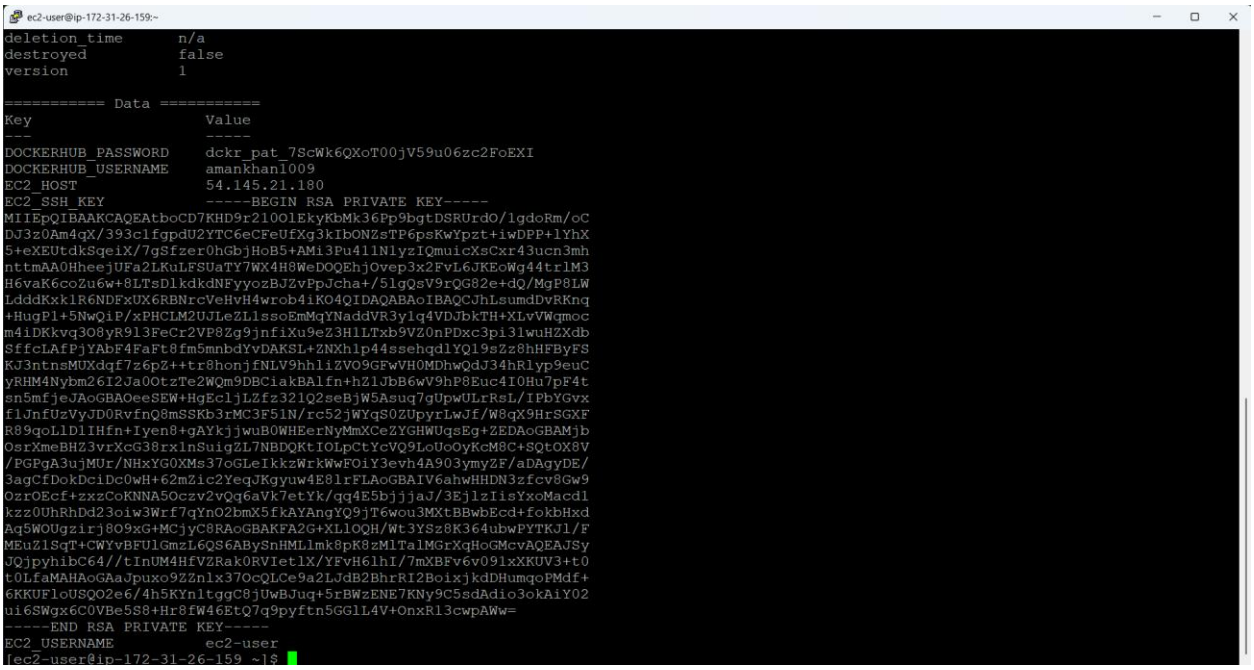


Figure 9: Secrets stored in Vault including DockerHub credentials and EC2 SSH key.

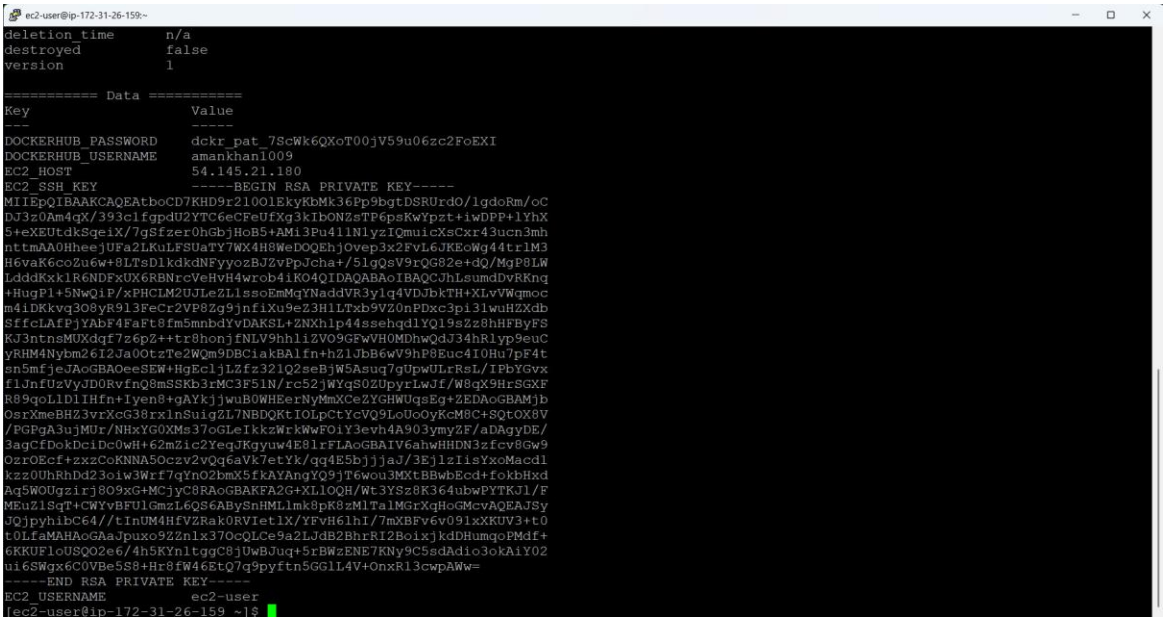
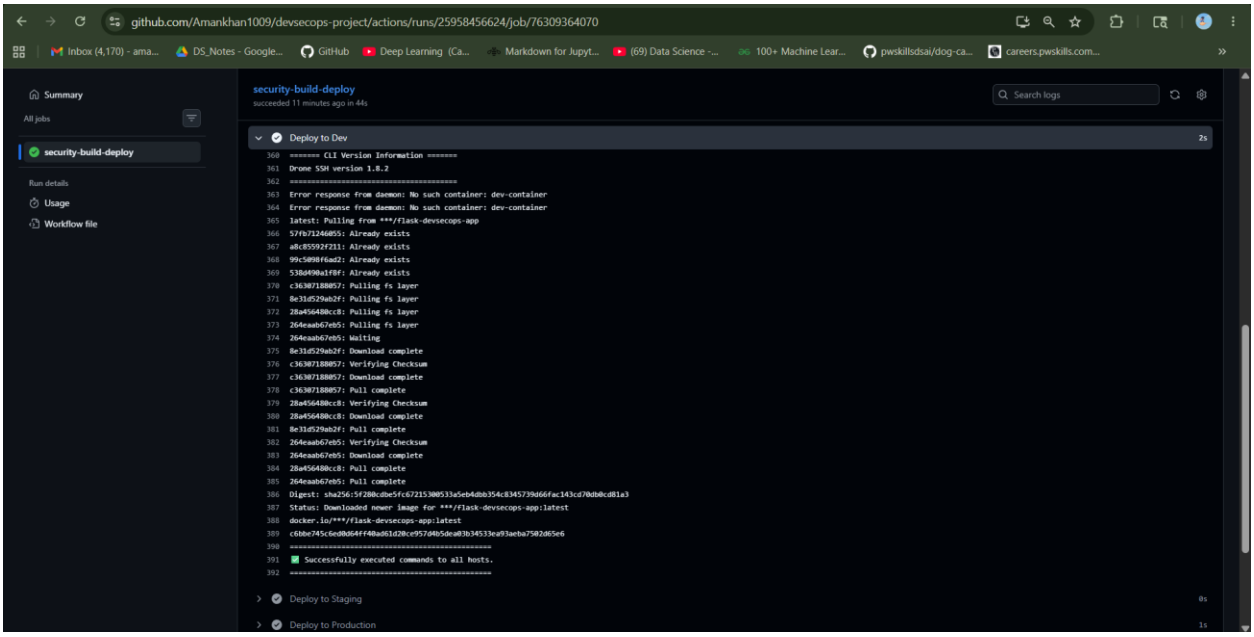


Figure 10: Full list of secrets stored in Vault including DOCKERHUB_PASSWORD, DOCKERHUB_USERNAME, EC2_HOST, EC2_SSH_KEY, and EC2_USERNAME.

9. Successful Pipeline Execution

After fixing all the issues including the Terraform directory, the security scan configurations, and the DockerHub authentication, the pipeline finally ran successfully. All stages passed including Gitleaks, Trivy, Checkov, DockerHub login, image push, and all three deployments.



9.1 Final Pipeline Run - Overview

The screenshot below shows the final successful pipeline run in GitHub Actions. All steps show a green checkmark, meaning everything passed without any errors.

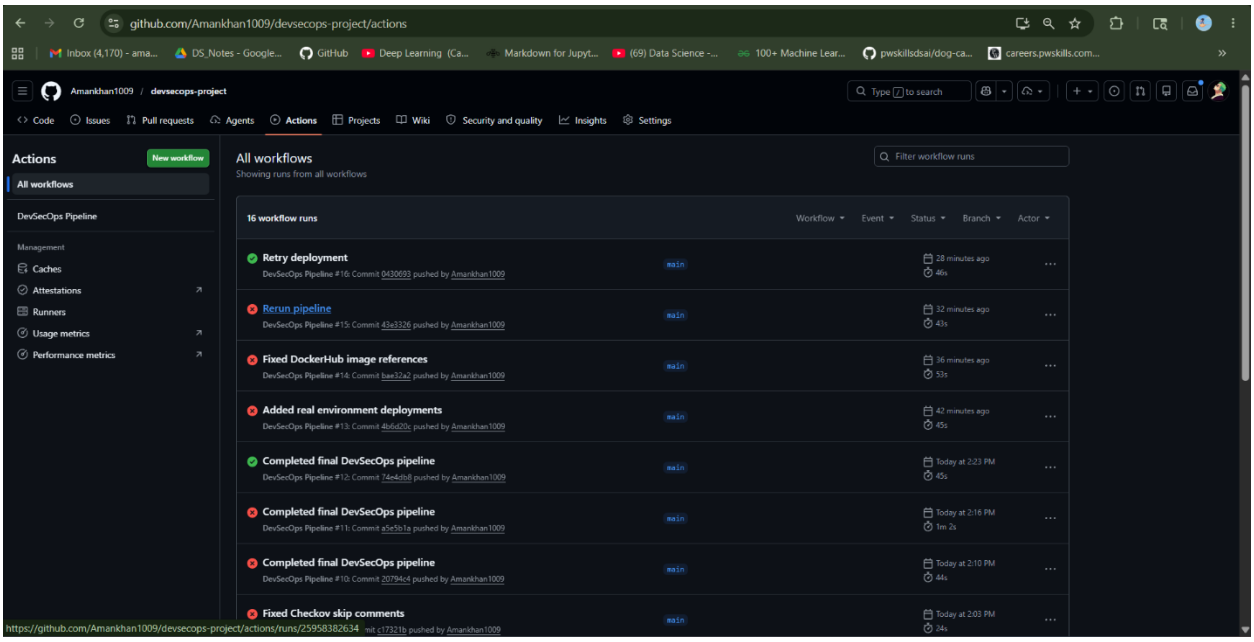


Figure 11: Final successful GitHub Actions pipeline run showing all stages passed.

9.2 All Pipeline Stages

The screenshot below shows all the individual pipeline stages including Setup Python, Install Dependencies, Install Gitleaks, Run Gitleaks Scan, Build Docker Image, Run Trivy Scan, Run Checkov,

DockerHub Login, Tag Docker Image, Push Docker Image, Deploy to Dev, Deploy to Staging, Deploy to Production, and post-cleanup steps. All of them have green checkmarks.

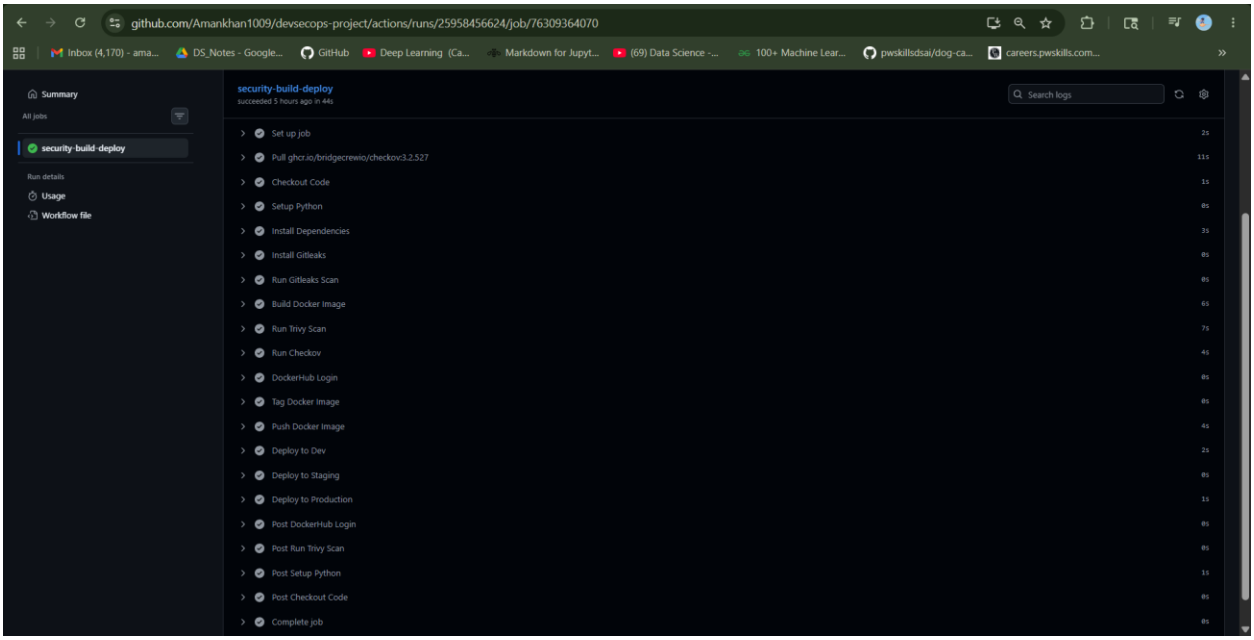


Figure 12: All pipeline stages completed successfully with green checkmarks.

9.3 Deployment Logs

The screenshot below shows the deployment logs for the Dev environment. The pipeline pulled the latest Docker image from DockerHub and started the container. The log shows "Successfully executed commands to all hosts" which confirms the deployment worked.

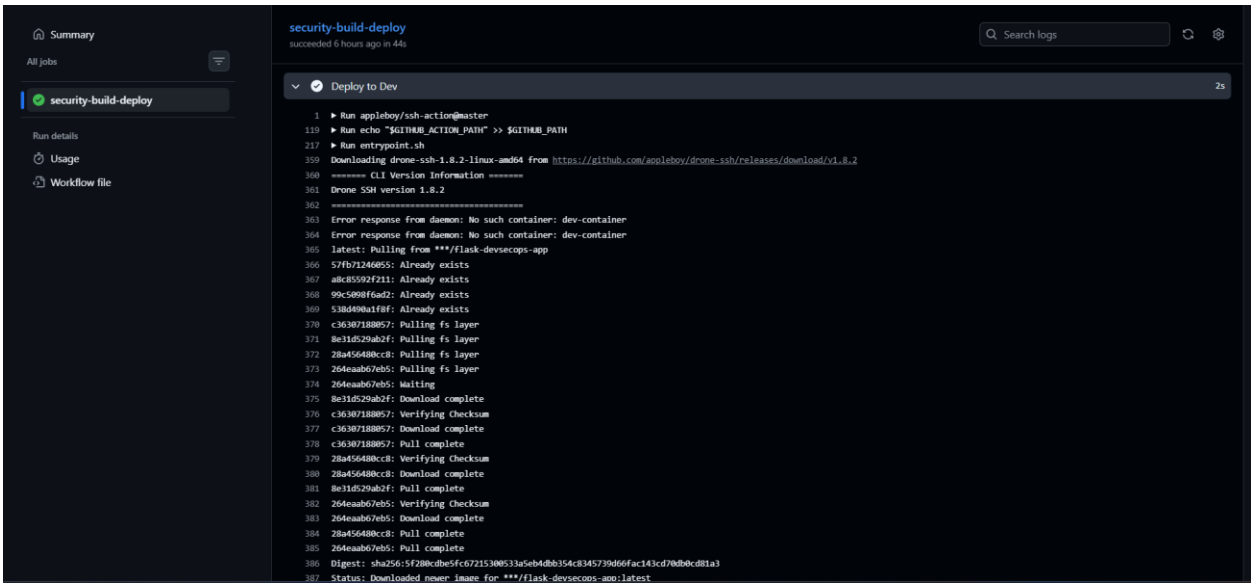


Figure 14: Deployment logs showing Docker image pulled and container started on EC2 on port5000.

10. Results - Application Running

After the pipeline completed, I verified that the application was running correctly on all three environments by opening the browser and visiting the EC2 IP address with each port number.

10.1 Dev Environment - Port 5000

The application was running on port 5000. The browser showed "DevSecOps Pipeline Running" which confirms the Dev environment deployment was successful.

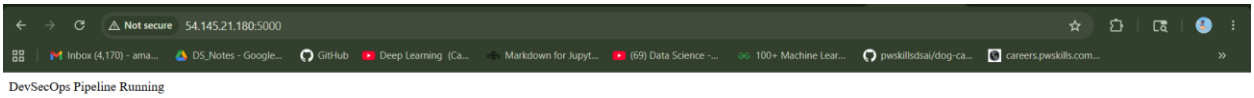


Figure 15: Application running on Dev environment at port 5000.

10.2 Staging Environment - Port 5001

The application was also running on port 5001. This confirms the Staging environment deployment was successful.

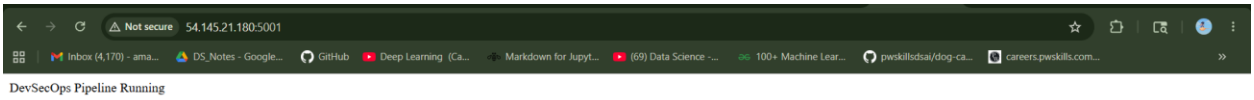


Figure 16: Application running on Staging environment at port 5001.

10.3 Production Environment - Port 5002

Finally the application was running on port 5002. This confirms the Production environment deployment was also successful. All three environments were working correctly after the pipeline finished.

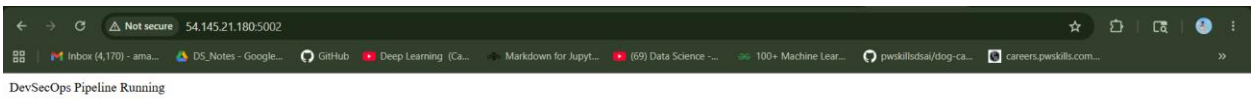


Figure 17: Application on Production environment at port 5002.

11. Challenges Faced

During this project I faced several challenges. Below I have explained each one and how I solved it.

During this project several technical and security-related challenges were encountered while building the DevSecOps CI/CD pipeline.

• Gitleaks license issue - The Gitleaks GitHub Action initially failed because the latest version required a license key. This was resolved by manually installing the standalone Gitleaks version inside the pipeline.

• Secret scanning failure - AWS credentials were intentionally added in the application file to test secret scanning. The pipeline failed successfully, proving that Gitleaks was detecting hardcoded secrets correctly.

• DockerHub authentication issue - The Docker image push failed because the DockerHub access token did not have proper permissions. The issue was fixed by generating a new token with Read and Write access.

• Vault configuration issue - While configuring HashiCorp Vault, authentication errors occurred because the VAULT_ADDR and VAULT_TOKEN variables were not set correctly. This was resolved by exporting the correct values before running Vault commands.

• Checkov IaC scanning failure - Initially, Checkov failed because the terraform/ directory and Terraform files were missing. The issue was resolved by creating the Terraform configuration files for AWS S3 infrastructure.

• Terraform policy violations - Checkov detected multiple security issues in the Terraform configuration related to encryption and bucket security. These issues were fixed by updating the Terraform code with secure configurations and policy skip comments.

• Deployment configuration challenges - Multiple changes were required in the GitHub Actions workflow to successfully deploy the application into Dev, Staging, and Production environments running on ports 5000, 5001, and 5002.

• Multiple pipeline re-runs - The pipeline had to be executed multiple times during debugging and troubleshooting. Each execution helped identify configuration and security issues more effectively.

12. Conclusion

This project gave me a good understanding of how DevSecOps pipelines work in real life. I learned how to set up GitHub Actions, write Docker files, scan images and code for security issues, manage secrets using Vault, and deploy applications automatically to multiple environments.

The most important thing I learned is that security should be part of every step in the pipeline, not something that is done at the end. By using tools like Gitleaks, Trivy, and Checkov inside the CI/CD pipeline, we can catch security problems early before they reach production.

By the end of the project, the pipeline was fully working. All 16 screenshots show the different stages of the project from the initial failures to the final successful deployment across all three environments.

13. References

- GitHub Actions Documentation - https://docs.github.com/actions
- Docker Documentation - https://www.docker.com/
- HashiCorp Vault Documentation - https://developer.hashicorp.com/vault
- Trivy Documentation - https://aquasecurity.github.io/trivy
- Checkov Documentation - https://www.checkov.io/
- Gitleaks Documentation - https://gitleaks.io/
- Terraform Documentation - https://developer.hashicorp.com/terraform
- AWS EC2 Documentation - https://docs.aws.amazon.com/ec2/